# TESTING

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

## Guideline 1: Anchor Assertions to the Specification, Not to the Code

- When prompting an AI assistant to generate test assertions, supply the specification as the basis for expected values.
- Never let the model derive expected values solely by reading the implementation under test.
- If the implementation is wrong, oracles built from it will also be wrong, and the tests will pass silently on the bug.

## Guideline 1: Reasoning

- The oracle problem: determining if observed behavior matches intended behavior predates LLMs by decades
- LLMs introduce a new variant: the model reads code and generates assertions capturing actual behavior, not what the spec requires
- Oracle deficiency, where there is no external reference for correctness, so the model trusts the implementation
- Oracle problem is the primary open weakness in current LLM-based test generation

## Guideline 1: Reasoning (continued)

- Practitioners repeatedly named hallucinated or incorrect assertions as their top concern with AI-generated tests
- Even when LLM-generated tests that compile and pass, incorrect assertions were a leading cause of low usability scores
- **Fix: give the model a specification to reason from, not just the code**

## Guideline 1: Key Practices (Prompting)

- Include a docstring, a requirements excerpt, or a worked example (input/output pair from a design doc) in the prompt alongside the code
- Explicitly instruct the model: "Derive the expected values from the specification above, not by tracing the code"
- For numeric computations, provide the mathematical definition or formula as the oracle source
- For string transformations, provide a reference example from documentation rather than asking the model to infer from the implementation

## Guideline 1: Key Practices (Reviewing the Result)

- For every assertion, ask: "Could this expected value have been read directly off the implementation?" If yes, verify it against the specification independently
- Look for circular oracles: assertions that rerun the same function or call a helper that wraps the same logic
- If the code under test contains a known bug, run generated tests against the buggy version first. A test that passes on buggy code is an oracle deficiency

## Guideline 2: Prompt for Boundary and Equivalence-Class Coverage Explicitly

- When asking an AI assistant to generate tests, list the boundary conditions and equivalence classes you expect to be covered directly in the prompt.
- Do not assume the model will apply structured test design techniques on its own.
- Without explicit guidance, LLMs default to happy-path examples.

## Guideline 2: Reasoning

- Classical insight: "test cases that explore boundary conditions have a higher payoff than cases that do not"
- Without explicit guidance, LLM-generated tests covered fewer branches, with the gap widest at boundary conditions
- Branch coverage remains the weakest dimension of AI-generated tests across 102 studies
- One prompt per execution path increased correct test generation by 5x for complex methods

## Guideline 2: Key Practices (Prompting)

- Before writing the prompt, list the equivalence classes and boundary values on paper (valid range, below minimum, above maximum, empty, null/None, duplicates)
- Paste that list verbatim into the prompt as a bulleted specification of what cases to cover
- For numeric inputs, always name the boundary values explicitly (e.g., test with 0, 1, 99, 100, 101 if the valid range is 1 to 100)
- For strings, always include empty string, exactly minimum length, exactly maximum length, and at least one character outside the allowed set

## Guideline 2: Key Practices (Reviewing the Result)

- Check that boundary values are tested at the boundary itself, not just somewhere in the valid range
- Confirm that invalid-class tests assert the rejection (exception raised or False returned), not just that the code runs without crashing

## Guideline 3: Tell the Model Which Dependencies to Mock and What Behavior to Stub

- When the unit under test calls external dependencies (databases, HTTP APIs, file systems, clocks), explicitly name each dependency in the prompt.
- Specify whether it should be mocked or stubbed, and describe the behavior each test variant needs the stub to exhibit.
- Do not let the model decide on its own whether to isolate the dependency or call it for real.

## Guideline 3: Reasoning

- Principle: a unit test must test exactly one unit in isolation, with all dependencies replaced by controlled substitutes
- Without explicit instructions, LLMs violate this in 3 ways: omit mocking entirely, generate mocks with unrealistic behaviors, or mock the wrong boundary
- A static analysis pre-pass identifying which types require mocking was necessary for usable tests on enterprise code
- LLMs generate reasonable mocks when given explicit instructions, but fail to do so reliably otherwise
- Dependency handling remains an open challenge; developers cannot rely on the model to solve it autonomously

## Guideline 3: Key Practices (Prompting)

- List every external dependency the method touches: database sessions, HTTP clients, file handles, clocks, random number generators
- For each dependency, state which mocking framework to use and at which point in the call chain to insert the mock
- For each test variant, describe stub behavior: "For success, stub the query to return [{'id': 1}]. For failure, stub it to raise ConnectionError"
- Ask the model to assert on the mock interactions themselves (e.g., assert the query was called exactly once with these arguments)

## Guideline 3: Key Practices (Reviewing the Result)

- Confirm no generated test makes a real network call, writes to the real filesystem, or queries a real database. Run tests without internet access as a quick check
- Check that mock return values are realistic enough to exercise the full logic (a stub returning None when the unit expects a list will crash before reaching the assertion)
- Verify that the mock sits at the edge of the class under test, not patching something inside it